2025 IEEE International Conference on Robotics and Automation (ICRA)
May 19-23, 2025. Atlanta, USA

Multi-Agent Obstacle Avoidance using Velocity Obstacles and Control Barrier Functions

Alejandro Sánchez Roncero, Rafael I. Cabral Muchacho and Petter Ögren

***Abstract*— Velocity Obstacles (VO) methods form a paradigm for collision avoidance strategies among moving obstacles and agents. While VO methods perform well in simple multi-agent environments, they do not guarantee safety and can show overly conservative behavior in common situations. In this paper, we propose to combine a VO strategy for guidance with a Control Barrier Function approach for safety, which overcomes the overly conservative behavior of VOs and formally guarantees safety. We validate our method in a baseline comparison study, using second-order integrator and car-like dynamics. Results support that our method outperforms the baselines with respect to path smoothness, collision avoidance, and success rates.**

I. Introduction

In this paper we propose a combination of Velocity Obstacles (VO) and Control Barrier Functions (CBF) for multi-agent collision avoidance. We use the classical CBF formulation of continuously solving an optimization problem to get a control input that is close to a desired one, while still guaranteeing safety. Instead of treating the VO as a constraint in this optimization, as [1], we consider it in the objective function, as suggested in [2], while keeping a less restrictive CBF as a constraint to guarantee safety.

Multi-agent collision avoidance is a well-studied problem with important applications in automated warehouses, autonomous driving, and airborne drone delivery systems. The idea of VO was first proposed in [3], with a number of refinements in e.g. [1], [2], [4], and a nice recent survey in [5]. The key idea is to make the assumption that all other agents, at least temporarily, will keep a constant velocity, make sure your own velocity is such that no collisions occur, and keep repeating this as the world changes. This family of approaches have been shown to be very efficient in handling challenging scenarios with many agents involved. However, as was noted in [2], the approach might be overly conservative in some scenarios, where it leads to the conclusion that there are no admissible velocity choices left.

A standard way of resolving this is to introduce a time horizon, where only potential collisions within that horizon are considered. This might lead to sudden changes in the control, and it was argued in [2] that it makes more sense to move the VO into the objective of an optimization,

This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation. The authors are with the Robotics, Perception and Learning Lab., School of Electrical Engineering and Computer Science, Royal Institute of Technology (KTH), SE-100 44 Stockholm, Sweden alesr@kth.se, ricm@kth.se, petter@kth.se Digital Object Identifier (DOI): see top of this page. Code: https://github.com/KTH-RPL/MA-CBF-VO Video: https://youtu.be/Ox8v2s17gLw

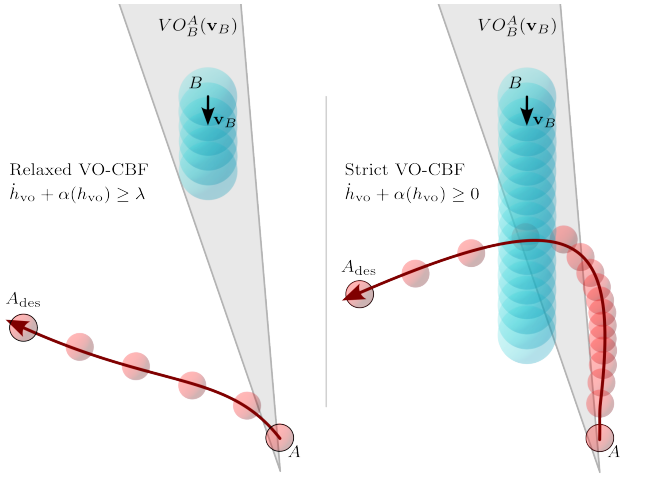

Fig. 1. An illustration showing that strict VO-based constraints can be overly conservative. Two red agents A aim for a destination A_{des} while avoiding the obstacle B: the one with a relaxed constraint (left) follows a direct path, while the one with a strict constraint (right) cannot turn in front of the obstacle (whose initial VO is shown in light gray) as this would imply at some point selecting a velocity inside the VO. In more crowded scenarios, such strict constraints can severely limit available options.

where actions leading to collisions are penalized, with higher weight given to more urgent cases. However, none of the classical VO papers include safety guarantees with respect to collisions. This problem is underlined by the numerical investigations of [6], where none of the VO approaches manages to avoid collisions in all scenarios.

An efficient tool for guaranteeing safety in autonomous robotics is CBFs [7]. The central idea in CBFs is to formulate the safety constraint in terms of a scalar function, and then control the time derivative of that function, in a way similar to control Lyapunov functions [8] so that the system never reaches the un-safe region.

Combining VO and CBFs is a natural idea, and has been investigated in [1], [9], [10]. In these papers, the VO was included in the constraints of the CBF. However, as noted by [2], see above, this might exclude a large part of the available control options in a scenario with many agents.

The main contribution of this paper is that we combine the ideas of [1] and [2] by merging CBFs and VO, but not by adding the VO as a hard constraint, but by using a slack variable to bring the VO into the objective function, while keeping another, less restrictive CBF constraint to guarantee safety.

The outline of the paper is as follows. In Section II we provide a brief background on CBFs and VO, and related

2025 IEEE International Conference on Robotics and Automation (ICRA) | 979-8-3315-4139-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICRA55743.2025.11127277

work within the area can be found in Section III. Then, in Section IV we present the proposed approach and the theoretical safety guarantees. The simulation results can be found in Section V, and conclusions are summarized in Section VI.

II. Background

In this section, we describe the two approaches that we seek to combine. First, we review the core result of CBFs, and then the VO approach. Finally, we briefly describe the motion models used in the examples.

A. Control Barrier Functions

CBFs have their roots in Lyapunov Theory [8], and an overview can be found in [7]. Let the system dynamics be control affine, i.e.,

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}, \tag{1}$$

where $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{u} \in \mathbb{R}^m$, $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^n$ and $\mathbf{g} : \mathbb{R}^n \to \mathbb{R}^{n\times m}$.

Let the safe set be given by $\mathcal{C} = \{\mathbf{x} : h(\mathbf{x}) \geq 0\}$, where h depends only on the state and has to satisfy some properties. Then, given the system dynamics $\mathbf{f}(\mathbf{x}, \mathbf{u})$, if the set

$$K = \{\mathbf{u} \in U : \frac{dh}{d\mathbf{x}}(\mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}) \geq -\alpha(h(\mathbf{x}))\}, \tag{2}$$

is non-empty for all $\mathbf{x}$, and if we choose controls $\mathbf{u}$ inside K, we are guaranteed to stay in the safe set $\mathbf{x} \in \mathcal{C}$, see [7]. The function α is a so-called class $\mathcal{K}$ function, that is $\alpha : \mathbb{R}_+ \to \mathbb{R}_+, \alpha(0) = 0$, and α is strictly monotonic increasing, see Theorem 2 in [7]. Also note that sometimes the Lie-derivative notation is used, where

$$\frac{dh}{d\mathbf{x}}\dot{\mathbf{x}} = \frac{dh}{d\mathbf{x}}(\mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}) = L_{\mathbf{f}}h(\mathbf{x}) + L_{\mathbf{g}}h(\mathbf{x})\mathbf{u}. \tag{3}$$

If some desired control is given by $\mathbf{u}_{\text{des}} = \mathbf{k}(\mathbf{x})$, we can formulate the following optimization problem to find a control $\mathbf{u}$ that is close to $\mathbf{k}(\mathbf{x})$ while still inside K, thereby keeping the state inside the safe set $\mathcal{C}$, [7]

$$\mathbf{u}(\mathbf{x}) = \underset{\mathbf{u}}{\operatorname{argmin}} \quad \frac{1}{2}||\mathbf{u} - \mathbf{k}(\mathbf{x})||^2 \tag{4}$$

$$\text{s.t.} \quad \mathbf{u} \in K. \tag{5}$$

The key observation about the problem above is that it is a so-called Quadratic Programming (QP) problem (Linear constraints and quadratic objective function) which can be solved to optimality very efficiently, allowing the QP-solution to be part of a closed control loop.

B. Velocity Obstacles

The idea of Velocity Obstacles (VO) was introduced in [3] and refined in e.g. [1], [2], [4]. The key idea is illustrated in Figure 2. If agent B is moving with fixed velocity, we can study the avoidance problem from the perspective of agent A. If agent A chooses the same velocity as agent B (at the lower tip of the darker cone) the relative distance will not change. If a component is then added moving towards B, there will be a collision at some point. The higher up in the dark cone, the sooner the collision will happen (we might think of the lower tip as a collision at $t = \infty$).

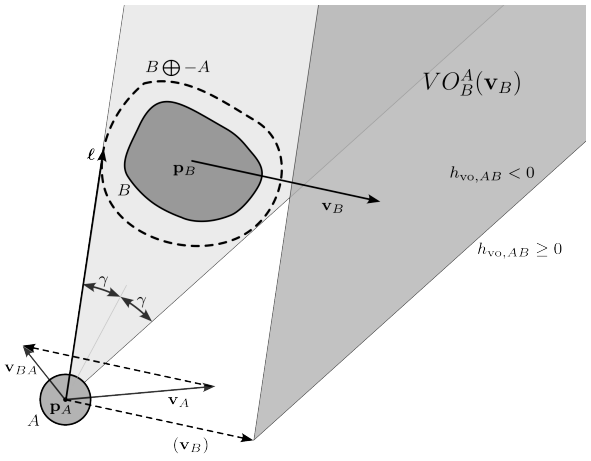

Fig. 2. Illustration of the key idea and notation used in Velocity Obstacle methods. If agent A chooses a velocity $\mathbf{v}_A$ outside the dark cone $VO_B^A(\mathbf{v}_B)$, it will not collide with agent B.

Formally, allowing arbitrary shapes for A and B by using the $\oplus$-notation, we can describe the concept above as follows. First, we define some set operations for $A, B \subset \mathbb{R}^n$ and $\mathbf{p}, \mathbf{v} \in \mathbb{R}^n$

$$A \oplus B = \{\mathbf{a} + \mathbf{b} \mid \mathbf{a} \in A, \mathbf{b} \in B\} \tag{6}$$

$$-A = \{-\mathbf{a} \mid \mathbf{a} \in A\} \tag{7}$$

$$\tau(\mathbf{p}, \mathbf{v}) = \{\mathbf{p} + t\mathbf{v} \mid t \geq 0\}, \tag{8}$$

where $\tau(\cdot)$ maps a state $(\mathbf{p}, \mathbf{v})$ to its future trajectory assuming constant velocity. Let two agents have positions $\mathbf{p}_A, \mathbf{p}_B$ and velocities $\mathbf{v}_A, \mathbf{v}_B$, all in $\mathbb{R}^n$. The velocity obstacle caused by B from the perspective of A is thus $VO_B^A(\mathbf{v}_B) = \{\mathbf{v}_A \mid \tau(\mathbf{p}_A, \mathbf{v}_A - \mathbf{v}_B) \cap B \oplus -A \neq \emptyset\}$.

C. Vehicle motion models used in the simulations

Here we describe the different model dynamics considered in this work. It should be noted that other models might equally work and have not been considered here.

For the double integrator in 2 dimensions we have $\mathbf{x} = (p_1, p_2, v_1, v_2)^T \in \mathbb{R}^4$ and $\mathbf{u} \in \mathbb{R}^2$, giving the affine form

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}, \tag{9}$$

with

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} v_1 \\ v_2 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{g}(\mathbf{x}) = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}. \tag{10}$$

For the car we have

$$\dot{p}_1 = v\cos\theta \tag{11}$$

$$\dot{p}_2 = v\sin\theta \tag{12}$$

$$\dot{\theta} = v/L\tan\phi \tag{13}$$

$$\dot{v} = a, \tag{14}$$

where p_1, p_2 is the position, θ represents the orientation of the car, v is the speed, L is the wheelbase distance, $\tan\phi$ is the steering control, and a is the acceleration control. Writing the above in the affine form $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}$, with $\mathbf{x} = (p_1, p_2, \theta, v)$, and $\mathbf{u} = (\tan\phi, a)$ we get

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} v\cos\theta \\ v\sin\theta \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{g}(\mathbf{x}) = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ v/L & 0 \\ 0 & 1 \end{bmatrix}. \tag{15}$$

Note that we treat $\tan\phi$ as the control variable to avoid having a nonlinear expression in $\mathbf{u}$.

III. Related Work

Collision avoidance in multi-agent systems is a vast field in itself, with subfields such as path planning [11], model predictive control (MPC) [12], [13] and artificial potential functions [14]. This paper is about merging the ideas of VO and CBFs so we will focus on the description of related work in these two areas.

The idea of velocity obstacles was introduced in [3] and extended in a number of papers [4], [15]–[17], that have refined and developed the concept since. One identified issue with VO was the oscillation problem. If two agents are facing each other, then they might first select a velocity that avoids collision. But in the next timestep, they might conclude that no collision is imminent, given the current velocity of the other agent, and then (both) turn back to their initial velocities. This will create an oscillation and works such as the Reciprocal Velocity Obstacle (RVO) [15] were proposed to resolve it. The RVO was later extended into the Hybrid RVO by [16] reducing the problem further, by making the velocity obstacles asymmetric, and built upon in [4], [17] where the problem is reduced to a linear programming problem that can be efficiently solved. In a paper on combining VO with MPC [13], the idea of including VO as constraints in the short-horizon motion planning problem was explored.

The work that lies closest to our approach within the VO literature is [2], where a VO term is used as part of the objective. We believe that this is a very good design choice, but our approach goes beyond [2] in the sense that we include a CBF element (not relying on VO) that guarantees safety.

CBFs are a tool for guaranteeing safety in control systems. Early works include [18], [19], with an overview in [7], and an extension to higher order systems in [20]. In connection to the avoidance of static obstacles, the connection between CBFs and artificial potential functions was explored in [21].

The works closest to ours within the CBF literature are [10] and [1], where the idea of combining CBFs and VO is proposed. However, in both papers, the authors add the VO as a hard constraint, which can be overly restrictive, as observed in [2] and illustrated in Fig. 1. Thus our approach goes beyond [1], [10] in the sense that the VO constraint is relaxed, and thereby essentially moved into the objective of the optimization, while another (non-VO) collision avoidance component is added to the CBF to provide the safety guarantees.

IV. Method

Instead of using velocity obstacles to directly ensure safety, we employ them for guidance through a relaxed inequality constraint and use a valid CBF to guarantee safe behavior, under the assumption that all agents seek to avoid collisions. Here, agent i denotes the agent for which we want to solve the optimization problem, and j indexes the dynamic obstacles. Altogether, the optimization problem for the agent i is formulated as

$$\mathbf{u}_i = \underset{\mathbf{u}_i}{\operatorname{argmin}} \quad k_u\|\mathbf{u}_i - \mathbf{u}_{\mathrm{ref},i}\|^2 + k_{\mathrm{vo}}\sum_{j=1}^{n_{\mathrm{obs}}} w_{ij}\lambda_{ij}^2 \tag{16}$$

$$\text{s.t.} \quad \dot{h}_{\mathrm{vo},ij} + \alpha_{\mathrm{vo}}(h_{\mathrm{vo},ij}) \geq \lambda_{ij}, \tag{17}$$

$$\dot{h}_{c,ij} + \alpha_c(h_{c,ij}) \geq 0, \tag{18}$$

$$\mathbf{u}_i \in \mathcal{U}, \tag{19}$$

where $\mathcal{U} = \{\mathbf{u} \in \mathbb{R}^m \mid \|\mathbf{u}\| \leq u_{\max}\}$ represents the valid input set. In this section we describe in detail the functions and variables composing the above optimization problem, starting with the objective function (16) and then finishing with the constraints (17)−(19) guaranteeing safety.

A. Objective Function

The objective function for each agent is composed of a goal attractive component J_{goal} and a velocity-obstacle guidance component J_{vo}. The goal attractive component is defined as the squared norm of the difference between the optimization variable to a reference input

$$J_{\mathrm{goal},i} = \|\mathbf{u}_i - \mathbf{u}_{\mathrm{ref},i}\|^2, \tag{20}$$

where the reference input $\mathbf{u}_{\mathrm{ref}}$ is given or computed by a known and state-dependent policy. In a scenario that is challenging with respect to the static obstacles, this would include planning a path that leads to the goal and then tracking this path. The path tracking control would then be $\mathbf{u}_{\mathrm{ref}}$.

The velocity obstacle component is inspired by the VO-CBF constraint [9]

$$h_{\mathrm{vo},ij}(\mathbf{x}) = \mathbf{p}_{ij}^T\mathbf{v}_{ij} + \|\mathbf{p}_{ij}\|\|\mathbf{v}_{ij}\|\cos(\gamma_{ij}) \tag{21}$$

$$\dot{h}_{\mathrm{vo},ij} \geq -\alpha_{\mathrm{vo}}(h_{\mathrm{vo},ij}), \tag{22}$$

where $\mathbf{p}_{ij} = \mathbf{p}_j - \mathbf{p}_i$ and $\mathbf{v}_{ij} = \mathbf{v}_j - \mathbf{v}_i$ represent the relative position and velocity vectors from agent i to j respectively, the angle γ is the semi-angle of the velocity obstacle cone with $\cos(\gamma_{ij}) = \hat{\mathbf{p}}_{ij}^T\hat{\boldsymbol{\ell}}_{ij}$, where *hat* denotes the normalized vector. The vector $\boldsymbol{\ell}_{ij}$ points from the i object's center, to a point on the augmented j object's surface and on the corresponding cone. The notation is also visualized in Figure 2. The time derivative of $h_{\mathrm{vo},ij}$, in (21), removing object subscripts, is given by

$$\begin{aligned}\dot{h}_{\mathrm{vo}} &= \dot{\mathbf{p}}^T\mathbf{v} + \mathbf{p}^T\dot{\mathbf{v}} + \dot{\mathbf{v}}^T\hat{\mathbf{v}}\mathbf{p}^T\hat{\boldsymbol{\ell}} + \|\mathbf{v}\|\dot{\mathbf{p}}^T\hat{\boldsymbol{\ell}} + \|\mathbf{v}\|\mathbf{p}^T\dot{\hat{\boldsymbol{\ell}}} \\ &= \mathbf{u}^T(\mathbf{p} + \hat{\mathbf{v}}\mathbf{p}^T\hat{\boldsymbol{\ell}}) + \|\mathbf{v}\|(\mathbf{v}^T\hat{\boldsymbol{\ell}} + \mathbf{p}^T\dot{\hat{\boldsymbol{\ell}}} + \|\mathbf{v}\|),\end{aligned} \tag{23}$$

which is linear in acceleration $\mathbf{u}$ for the double integrator system.

Intuitively, the cone CBF constraint (22) keeps the relative vector $\mathbf{v}_{ji}$ outside of the Velocity Obstacle cone. Thus, satisfying (22) keeps the system safe, but it is overly constraining in many real scenarios, as pointed out in [2], and illustrated in Figure 1. As can be seen, this problem is significant in the case of a single obstacle and gets even worse with multiple obstacles around, potentially making it very difficult to find a velocity outside of the joint set of Velocity Obstacles. The basic formulation used in [10] would result in the agent not finding any solution to the set of inequalities (other than the trivial 0 if its velocity is 0). To overcome this problem, we incorporate the VO as part of the objective by introducing an auxiliary (slack) variable λ in Equation (22), giving

$$\dot{h}_{\mathrm{vo},ij} + \alpha_{\mathrm{vo}}(h_{\mathrm{vo},ij}) \geq \lambda_{ij}. \tag{24}$$

and then penalizing this slack variable in the objective as

$$J_{\mathrm{vo},ij} = \lambda_{ij}^2. \tag{25}$$

The total objective function now becomes a weighted sum of the objective components,

$$J_i = k_u J_{\mathrm{goal},i} + k_{\mathrm{vo}} \sum_{j=1}^{n_{\mathrm{obs}}} w_{ij} J_{\mathrm{vo},ij}, \tag{26}$$

with constant positive scaling factors $k_u, k_{\mathrm{vo}} \in \mathbb{R}_+$. The weights are computed as a function of an approximate time-to-collision $T_{col,ij}$ (based on the linear extrapolation of positions) between objects i and j

$$w_{ij} = \frac{1}{T_{\mathrm{col},ij}}. \tag{27}$$

If objects i and j do not collide according to the used model, then $T_{\mathrm{col},ij} \to \infty$ and we set $w_{ij} = 0$. When using a linear model for extrapolation, the time-to-collision and therefore the weights are symmetric, i.e., $w_{ij} = w_{ji}$. The choice of an inverse time-to-collision for weighting encodes that an agent should dedicate more effort to avoid objects with which it would collide sooner if no further action is taken.

B. Safety-Critical Constraint

The safety of agents is formally guaranteed by a CBF based on position and velocity for collision avoidance [22], [23]. The function h_c implicitly describes safe states as states at which the distance to an obstacle is higher than the braking distance at maximum acceleration or braking effort.

The variable d_{ij} represents the minimum distance between the i and j objects. We assume objects are convex, and therefore each of their distance functions is continuously differentiable. This assumption is not restrictive as continuously differentiable approximations of distance functions are readily available [24]–[26]. A valid CBF for double integrator systems with saturated input $\|\mathbf{u}\| \leq u_{\max}$ is

$$h_{c,ij} = d_{ij} - \delta - \frac{\nu_{ij}^2}{2u_{\max}} \tag{28}$$

$$\nu_{ij} = \min\left(0, \mathbf{v}_{ij}^T \hat{\mathbf{p}}_{ij}\right), \tag{29}$$

where $\delta \geq 0$ is a safety margin, $\hat{\mathbf{p}}_{ij}$ is the normalized vector between the agent and the closest point on object j, which makes ν_{ij} the relative velocity of the other object projected onto this direction with large negative ν_{ij} indicating a potential problem.

In the case where $\mathbf{v}_{ij}^T \hat{\mathbf{p}}_{ij} \leq 0$ we get

$$\dot{h}_{c,ij} = \dot{d}_{ij} - \frac{\nu_{ij}}{u_{\max}}(\mathbf{u}_{ij}^T \hat{\mathbf{p}}_{ij} + \mathbf{v}_{ij}^T \dot{\hat{\mathbf{p}}}_{ij}) \tag{30}$$

i.e., time derivative of the function (28) is linear in the acceleration. If $\mathbf{v}_{ij}^T \hat{\mathbf{p}}_{ij} > 0$ we get $\dot{h}_{c,ij} = \dot{d}_{ij} = \mathbf{v}_{ij}^T \hat{\mathbf{p}}_{ij} > 0$. Either way, the corresponding linear inequality constraint to guarantee safety will be

$$\dot{h}_{c,ij}(\mathbf{u}_{ij}) + \alpha_c(h_{c,ij}) \geq 0, \tag{31}$$

where α_c is a class $\mathcal{K}$ function. Note that if $\mathbf{v}_{ij}^T \hat{\mathbf{p}}_{ij} > 0$, Equation (31) is independent of $\mathbf{u}_{ij}$ and always satisfied.

V. Experiments

In this section, we evaluate our method in multi-agent scenarios, through a baseline comparison study and in a validation scenario with car-like dynamics. We show that

1) the formal safety guarantee translates into safe agent behavior in practice,
2) the VO-based component successfully guides the agents through the tasks,
3) our method outperforms baselines w.r.t. path smoothness, collision avoidance, and success rates.

Code[1] and videos[2] of the experiments are available online.

A. Setup

Our approach is implemented in MATLAB and Simulink3D. For simplicity, agents are modeled as spheres with constant radii. We evaluate both 2nd-order integrator dynamics and car-like dynamics, given by Equations (10), (15) respectively. We assume full knowledge of the environment, including the positions, velocities, and dimensions of the agents. While the approach can be extended to handle more general agent shapes or uncertainties in position and velocity, see [16], we focus here on deterministic settings.

We run simulations with fixed timesteps, with the control output calculated for each agent at each step. The states are updated simultaneously for all agents. For our model, we define constraints and solve the optimization problem using MATLAB's linear least-squares solver at each timestep. The acceleration is constrained during optimization, and forward Euler integration is used to propagate the resulting accelerations to compute velocities and positions. Further details on the simulation setup are provided in Table I.

B. Comparison to Baselines

1) Setup: As baselines, we used Velocity Obstacles (VO), Reciprocal Velocity Obstacles (RVO), Collision Cone CBF (hVO), and Optimal Velocity selection using Velocity Obstacle (OVVO). For implementation, we followed the methodology outlined in [2], [15], while focusing on the second-order integrator dynamics. To handle the fact that the other

[1] https://github.com/KTH-RPL/MA-CBF-VO
[2] https://youtu.be/Ox8v2s17gLw

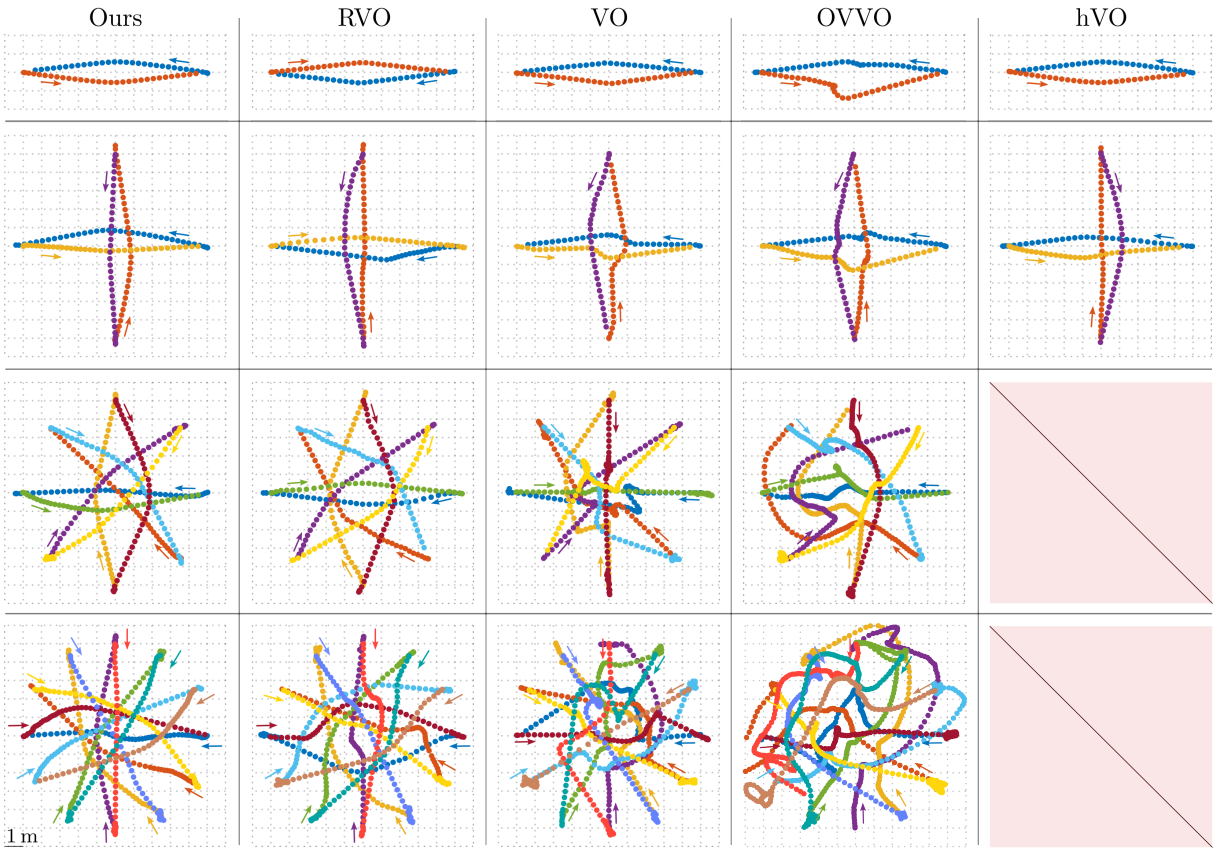

Fig. 3. Visualization of the resulting trajectories in the baseline comparison study. The arrows in each corresponding color describe the direction of motion of each agent. Our method (left-most column) remains stable and computes smooth trajectories for all evaluated scenarios. The hVO method encountered infeasibility (no available safe velocities) in the scenarios with 8 and 12 agents. The aspect ratio is equal for both dimensions and the scale is given by the bottom left reference.

approaches had no explicit motion model, with control $\mathbf{u}$, we did as follows. At each timestep, we define and sample a set of admissible velocities based on the maximum allowable acceleration and timestep. For VO and RVO, we select the one that minimizes (analogous to (16)) the penalty

$$J_{\text{VO,RVO}} = \frac{1}{T_{\text{col}}} + ||\mathbf{v} - \mathbf{v}_{\text{des}}||, \tag{32}$$

where T_{col} is the minimum time-to-collision among obstacles. For OVVO, the penalty is expressed as

$$J_{\text{OVVO}} = k_{tp} d_v^{-c_1} t_p^{-c_2} + k_{vd} ||\mathbf{v} - \mathbf{v}_{\text{des}}||, \tag{33}$$

where k_{tp}/k_{vd} balances deviating from the VO cone and following the desired control input, and c_1, c_2 are constants. d_v and t_p represent the clearance and pass time, closely related to a separation between obstacles and time-to-collision respectively (we refer the reader to [2] for further details). We recover the hVO formulation by setting k_{cone} and the slack variables (λ_i) to 0 while removing the safety-critical constraints.

The time-to-collision (T_{col}) is computed assuming constant velocity, solving for the time t such that

$$\tau(\mathbf{p}_A, \mathbf{v}'_A - \mathbf{v}_B) \cap B \oplus -A \quad \text{(VO)} \tag{34}$$

$$\tau(\mathbf{p}_A, 2 \cdot \mathbf{v}'_A - \mathbf{v}_A - \mathbf{v}_B) \cap B \oplus -A \quad \text{(RVO)}, \tag{35}$$

where $\mathbf{v}'_A$ is the desired velocity for agent A. The selected velocity is divided by the timestep to obtain the control input. All parameters of the models are manually tuned with minimal effort to evaluate qualitative performance. No optimization is applied. The control scheme is PD-based, using position error as the input to guide the agents toward their goals.

We evaluate our method in a circular formation scenario to compare its performance against the baselines. Agents are equidistantly positioned on a circle with zero initial velocity, and their goal is to reach the diametrically opposite point. This naturally creates conflicting trajectories, leading to potential collisions. As the system is deterministic, we add noise to the initial position to compute statistics. We examine cases with 2, 4, 8, and 12 agents, all governed by second-order integrator dynamics. Each scenario is run 10 times and statistics are reported.

2) Results: Results are summarized in Table II and visualized in Figure 3. Qualitatively, our method produces smoother trajectories compared to the baselines. While all methods perform well with few agents, increasing the number of agents resulted in greater challenges, particularly for VO, hVO, and OVVO, leading to numerous collisions or low success rates. RVO performs relatively well, but still could

Parameter	Second-order Int.	Car Dynamics
Simulation time	60 s	60 s
Timestep	10 ms	10 ms
α_{vo}	10	10
α_{c}	10	10
k_u	1	1
k_{vo}	1000	1
Preferred velocity (v_{pref})	1 m/s	-
Maximum velocity ($v_{\max}$)	2 m/s	10 m/s
Maximum acceleration ($u_{\max}$)	1 m/s^2	3 m/s^2
Maximum steering ($\tan(\phi)$)	-	2
Geometric tolerance	10 %	10 %
Goal position tolerance	0.5 m	1 m
Agent Radii	0.5 m	1 m
Car's characteristic length (L)	1 m	1 m
P coefficient	1	0.2
D coefficient	0.5	-
N sampling points	250	-
k_{tp}, k_{vd}, c_1, c_2	2, 1, 1, 1	-

TABLE I

MAIN PARAMETERS OF THE SIMULATION.

N.A	Method	S.R.	Collisions	Simulation Time (s)	Computation Time(ms)
2	Ours	1	0.00±0.00	15.28±0.06	4.86±1.09
	VO	1	0.00±0.00	15.18±0.08	3.84±0.70
	RVO	1	0.00±0.00	**15.13±0.08**	**3.64±0.50**
	hVO	1	0.00±0.00	15.25±0.08	5.00±0.00
	OVVO	1	0.00±0.00	17.66±0.34	13.86±1.11
4	Ours	1	0.00±0.00	17.39±0.76	**4.98±0.76**
	VO	1	0.00±0.00	17.31±1.36	10.82±0.60
	RVO	1	0.00±0.00	**15.77±0.84**	10.32±0.72
	hVO	0.8	0.00±0.00	14.45±6.17	5.60±0.00
	OVVO	1	0.00±0.00	30.62±8.22	28.24±2.30
8	Ours	1	0.00±0.00	21.2±0.82	**4.96±0.81**
	VO	1	0.80±1.40	27.28±7.78	12.70±4.73
	RVO	1	0.00±0.00	**19.76±1.40**	18.11±0.70
	hVO	0	0.00±0.00	0.73±0.22	6.99±0.26
	OVVO	0.6	3.50±4.50	29.69±16.43	48.09±3.13
12	Ours	1	0.00±0.00	25.38±1.91	**6.18±0.06**
	VO	1	15.00±6.78	33.24±5.37	17.30±6.37
	RVO	1	0.30±0.95	**23.43±1.98**	24.94±1.12
	hVO	0	0.00±0.00	0.65±0.11	7.90±0.60
	OVVO	0	13.80±5.63	29.80±26.06	70.40±3.68

TABLE II

RESULTS OF THE EXPERIMENTS. THE ENTRIES MARKED IN RED HAVE SIGNIFICANT FAILURES IN TERMS OF EITHER SUCCESS RATE (S.R., REACHING THE GOAL) OR COLLISIONS. OUT OF THE NON-RED ROWS, THE BEST IN TERMS OF SIMULATION TIME AND COMPUTATION TIME ARE WRITTEN IN BOLD FONT. N.A – NUMBER OF AGENTS. SIMULATION TIME – AVERAGE TOTAL EPISODE LENGTH. COMPUTATION TIME – PER ITERATION SOLVE AND SAMPLING TIME, RESPECTIVELY.

not completely avoid collisions, as seen in the 12-agent case. In contrast, our approach consistently yielded zero collisions, as predicted by the safety guarantees.

In terms of success rate, hVO fails at 8 and 12-agent scenarios. At the start of the simulations, all agents are in a safe state with zero velocity. This leads them to choose a velocity towards their goals. Thus, at the next timestep, all of them are facing each other with colliding velocities, and since the scenario is crowded and acceleration is constrained, no feasible solution is found to the problem without relaxing the constraints. In terms of completion time, RVO performed best, with our approach second, but never needing more than 10% extra time to finish.

Regarding computation time, solving the quadratic programming problem in our method proved faster and more efficient than the random sampling process used by the baselines. However, our approach did incur some overhead due to the need to compute constraints for all agents.

Thus, while our method is similar to RVO in overall performance, it provides safety guarantees. Moreover, the smoother trajectories might provide less wear and tear in hardware over time. RVO assumes a degree of cooperation among agents, whereas our method is more general and can be employed in decentralized systems. Since the constraints are linear, we get a Quadratic Programming problem (QP), keeping computational complexity unchanged—an important advantage for scaling to larger problems.

C. Car Dynamics Example

We extend the experiments to car-like dynamics, see Equation (15), maintaining the same methodology. Two scenarios are evaluated: one with 4 agents and another with 8 agents, where the agents are tasked to reach the diametrically opposite location. The results are consistent with the second-order integrator dynamics. As can be seen in Figure 4, the resulting trajectories are smooth, and the non-holonomic agents successfully adapted to each other.

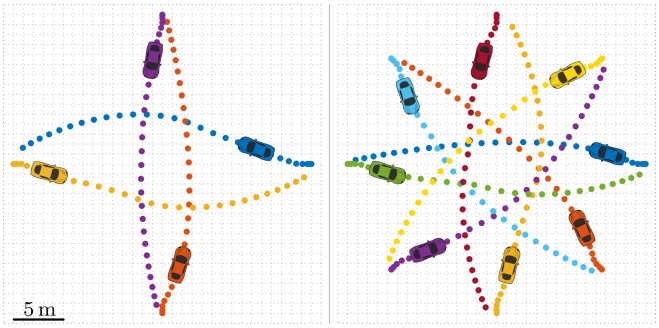

Fig. 4. Trajectories when applying the proposed approach to cars.

VI. CONCLUSION

In this paper we have shown how to combine CBFs with VO in a way that guarantees safety, and leverages the efficiency provided by the VO by including it in the objective of the optimization, overcoming the downside of being overly conservative that results from considering VOs as hard constraints. The simulation comparisons show that the proposed approach outperforms the alternatives in terms of path smoothness, success rates, and collision rates.

REFERENCES

[1] M. Tayal, R. Singh, J. Keshavan, and S. Kolathaya, "Control Barrier Functions in Dynamic UAVs for Kinematic Obstacle Avoidance: A

Collision Cone Approach," in *2024 American Control Conference (ACC)*. Toronto, ON, Canada: IEEE, Jul. 2024, pp. 3722–3727.
[2] M. Kim and J.-H. Oh, "Study on optimal velocity selection using velocity obstacle (OVVO) in dynamic and crowded environment," *Autonomous Robots*, vol. 40, no. 8, pp. 1459–1470, Dec. 2016.
[3] P. Fiorini and Z. Shiller, "Motion Planning in Dynamic Environments Using Velocity Obstacles," *The International Journal of Robotics Research*, vol. 17, no. 7, pp. 760–772, Jul. 1998.
[4] J. Van Den Berg, Ming Lin, and D. Manocha, "Reciprocal Velocity Obstacles for real-time multi-agent navigation," in *2008 IEEE International Conference on Robotics and Automation*. Pasadena, CA, USA: IEEE, May 2008, pp. 1928–1935.
[5] F. Vesentini, R. Muradore, and P. Fiorini, "A survey on Velocity Obstacle paradigm," *Robotics and Autonomous Systems*, vol. 174, p. 104645, Apr. 2024.
[6] J. A. Douthwaite, S. Zhao, and L. S. Mihaylova, "Velocity Obstacle Approaches for Multi-Agent Collision Avoidance," *Unmanned Systems*, vol. 07, no. 01, pp. 55–64, Jan. 2019.
[7] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control Barrier Functions: Theory and Applications," in *2019 18th European Control Conference (ECC)*, Jun. 2019, pp. 3420–3431.
[8] S. Sastry, *Nonlinear Systems: Analysis, Stability, and Control*. Springer Science & Business Media, 2013, vol. 10.
[9] M. Tayal and S. Kolathaya, "Polygonal Cone Control Barrier Functions (PolyC2BF) for Safe Navigation in Cluttered Environments," in *2024 European Control Conference (ECC)*, Jun. 2024, pp. 2212–2217.
[10] P. Thontepu, B. G. Goswami, M. Tayal, N. Singh, S. S. P I, S. S. M G, S. Sundaram, V. Katewa, and S. Kolathaya, "Collision Cone Control Barrier Functions for Kinematic Obstacle Avoidance in UGVs," in *2023 Ninth Indian Control Conference (ICC)*. Visakhapatnam, India: IEEE, Dec. 2023, pp. 293–298.
[11] V. R. Desaraju and J. P. How, "Decentralized path planning for multi-agent teams with complex constraints," *Autonomous Robots*, vol. 32, pp. 385–403, 2012.
[12] J. V. Frasch, A. Gray, M. Zanon, H. J. Ferreau, S. Sager, F. Borrelli, and M. Diehl, "An auto-generated nonlinear MPC algorithm for real-time obstacle avoidance of ground vehicles," in *2013 European Control Conference (ECC)*. IEEE, 2013, pp. 4136–4141.
[13] N. Piccinelli, F. Vesentini, and R. Muradore, "MPC Based Motion Planning For Mobile Robots Using Velocity Obstacle Paradigm," in *2023 European Control Conference (ECC)*, Jun. 2023, pp. 1–6.
[14] O. Khatib, "The potential field approach and operational space formulation in robot control," in *Adaptive and Learning Systems: Theory and Applications*. Springer, 1986, pp. 367–377.
[15] J. Van Den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance," in *Robotics Research: The 14th International Symposium ISRR*. Springer, 2011, pp. 3–19.
[16] J. Snape, J. Van Den Berg, S. J. Guy, and D. Manocha, "The hybrid reciprocal velocity obstacle," *IEEE Transactions on Robotics*, vol. 27, no. 4, pp. 696–706, 2011.
[17] J. Alonso-Mora, A. Breitenmoser, M. Rufli, P. Beardsley, and R. Siegwart, "Optimal reciprocal collision avoidance for multiple non-holonomic robots," in *Distributed Autonomous Robotic Systems: The 10th International Symposium*. Springer, 2013, pp. 203–216.
[18] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control Barrier Function Based Quadratic Programs for Safety Critical Systems," *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, Aug. 2017.
[19] P. Ogren, A. Backlund, T. Harryson, L. Kristensson, and P. Stensson, "Autonomous UCAV Strike Missions Using Behavior Control Lyapunov Functions," in *AIAA Guidance, Navigation, and Control*, 2006.
[20] W. Xiao and C. Belta, "Control Barrier Functions for Systems with High Relative Degree," in *2019 IEEE 58th Conference on Decision and Control (CDC)*. Nice, France: IEEE, Dec. 2019, pp. 474–479.
[21] A. Singletary, K. Klingebiel, J. Bourne, A. Browning, P. Tokumaru, and A. Ames, "Comparative Analysis of Control Barrier Functions and Artificial Potential Fields for Obstacle Avoidance," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Sep. 2021, pp. 8129–8136.
[22] A. Ghaffari, I. Abel, D. Ricketts, S. Lerner, and M. Krstić, "Safety verification using barrier certificates with application to double integrator with input saturation and zero-order hold," in *2018 Annual American Control Conference (ACC)*. IEEE, 2018, pp. 4664–4669.
[23] Y. Chen, M. Jankovic, M. Santillo, and A. D. Ames, "Backup control barrier functions: Formulation and comparative study," in *2021 60th IEEE Conference on Decision and Control (CDC)*. IEEE, 2021, pp. 6835–6841.
[24] R. I. C. Muchacho and F. T. Pokorny, "Adaptive distance functions via kelvin transformation," *arXiv preprint arXiv:2406.03200*, 2024.
[25] Y. Li, Y. Zhang, A. Razmjoo, and S. Calinon, "Representing robot geometry as distance fields: Applications to whole-body manipulation," in *Proc. IEEE Intl Conf. on Robotics and Automation (ICRA)*, 2024, pp. 15 351–15 357.
[26] P. Liu, K. Zhang, D. Tateo, S. Jauhri, J. Peters, and G. Chalvatzaki, "Regularized deep signed distance fields for reactive motion generation," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022, pp. 6673–6680.